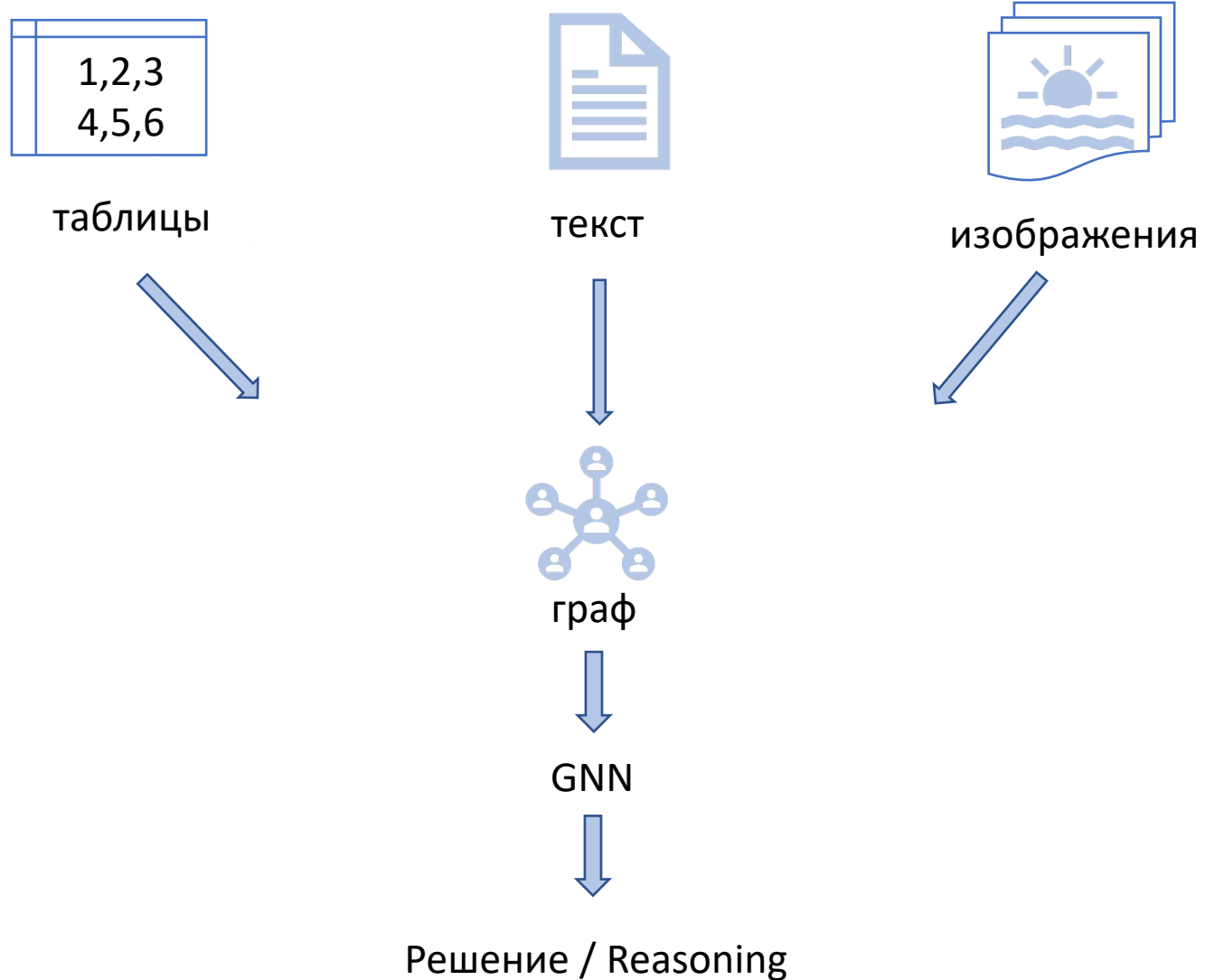


Графовые нейросети и их применение

Зачем индустрии GNN?

От данных — к моделированию связей

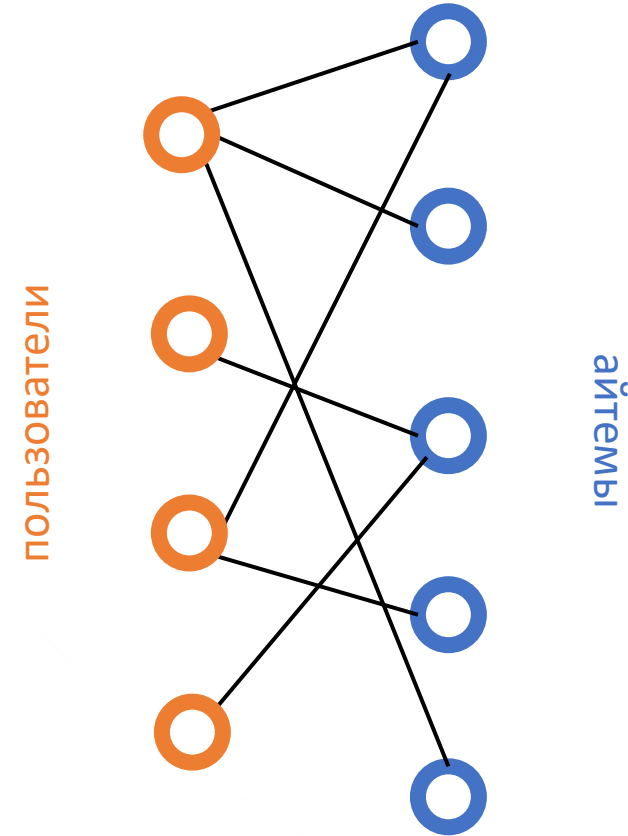


GNN в RecSys

Recomender System as a Graph

Рекомендательную систему можно представить в виде bipartite графа:

- Граф имеет два вида узлов:
 - Пользователи
 - Айтемы
- Ребра соединяют пользователей и айтомов
 - Индикаторы взаимодействия между пользователями и айтемами
 - Дополнительные фичи



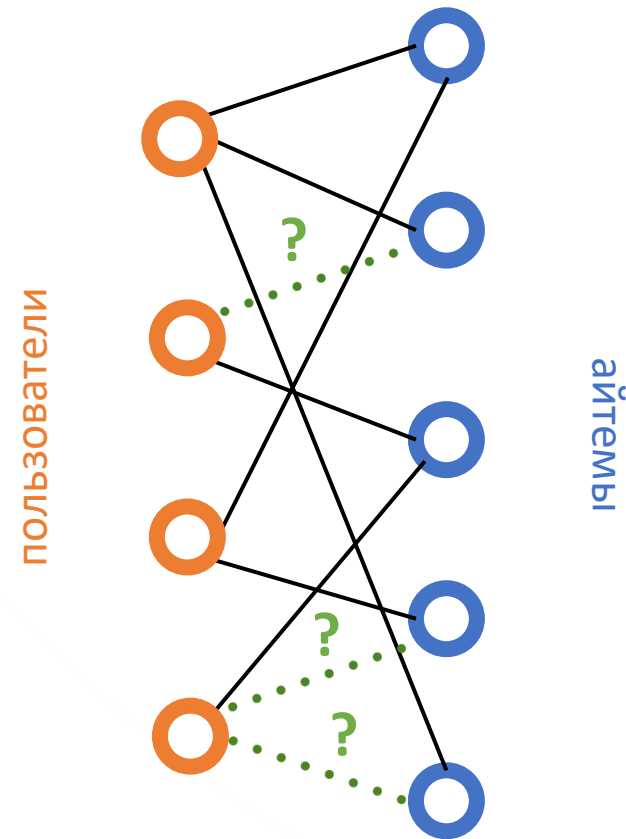
Recommendation task

Дано:

- История взаимодействия user-item

Задача:

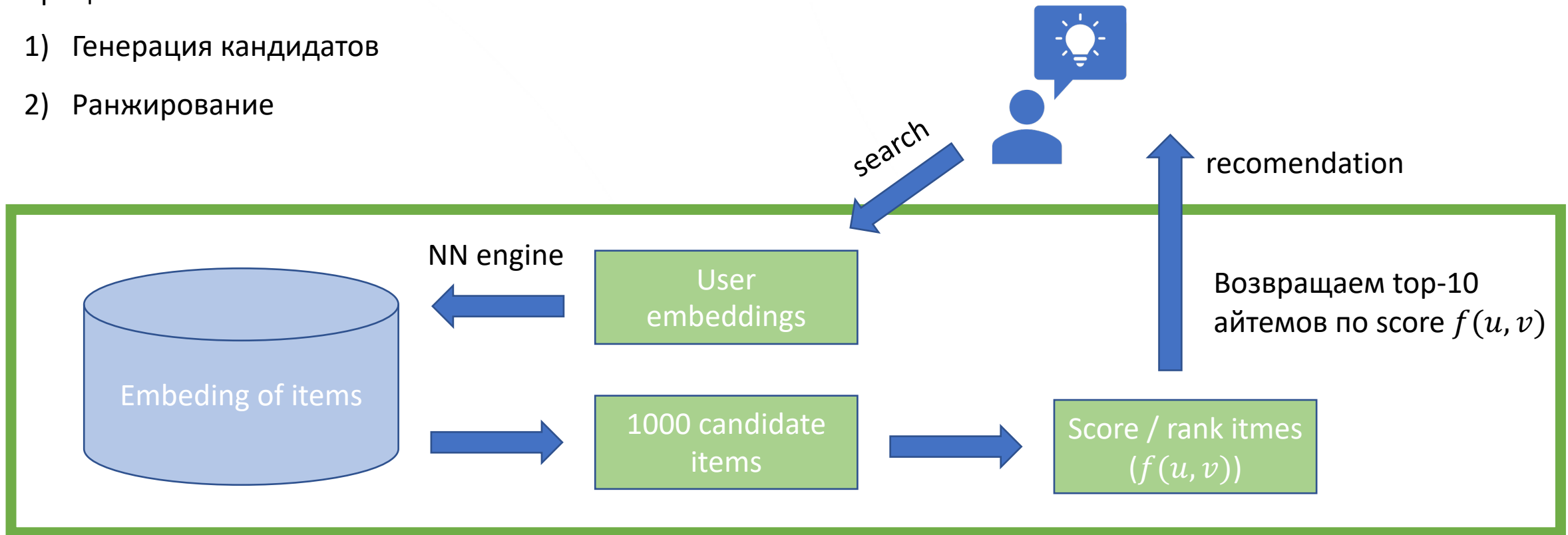
- Предсказываем новые айтемы, с которыми будет взаимодействовать пользователь в будущем
- Можно представить как задачу link prediction
 - Прогнозирование новых связей взаимодействия пользователя и товара на основе прошлых связей
- Для каждого $u \in U, v \in V$ нам нужно получить оценки $f(u, v)$



Modern Recommendation System

Рекомендательная система состоит из 2 основных процессов:

- 1) Генерация кандидатов
- 2) Ранжирование



Оценка

Метрика - Recall@K

доля реально интересных items в топ-K рекомендаций

Расчет метрики для одного пользователя

Recall@K for user u is $|P_u \cap R_u|/|P_u|$

Расчет итоговой метрики для всей системы:

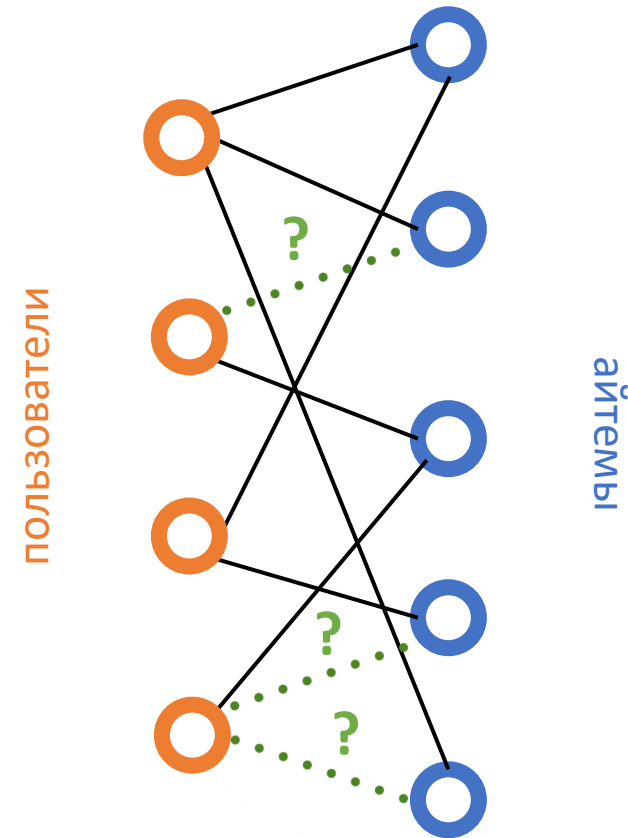
Среднее значение recall по всем пользователям



Embedding-Based models

Мы рассматриваем модели, основанные на эмбедингах, для оценки взаимодействий между пользователем и товаром.

- Без user / item фичей
- Используя encoders для users и items:
 - Для каждого $u \in U$, и $v \in V$ мы получаем обучаемый embedding $\mathbf{z}_u, \mathbf{z}_v \in \mathbb{R}^D$
- Score функция для users и items $f(u, v) \equiv \mathbf{z}_u^T \mathbf{z}_v$



Обучение

Обучение через surrogate-loss:

Почему напрямую не обучаем Recall@K

Recall@K — не дифференцируемый.

Поэтому вводятся surrogate-loss функции:

- **Binary loss**
- **BPR loss (Bayesian Personalized Ranking)** — особенно популярна в RecSys

Почему нужны GNN в RecSys

Ограничения обычных embedding-подходов

- Не используют **структуру графа**
- Не учитывают многошаговые зависимости
- Слабо обобщают в sparse-режиме

Нам нужна модель, которая:

- явно отражает структуру графа (помимо неявного отражения через целевую функцию обучения)
- отражает структуру графа высокого порядка (помимо структуры связности ребер первого порядка)

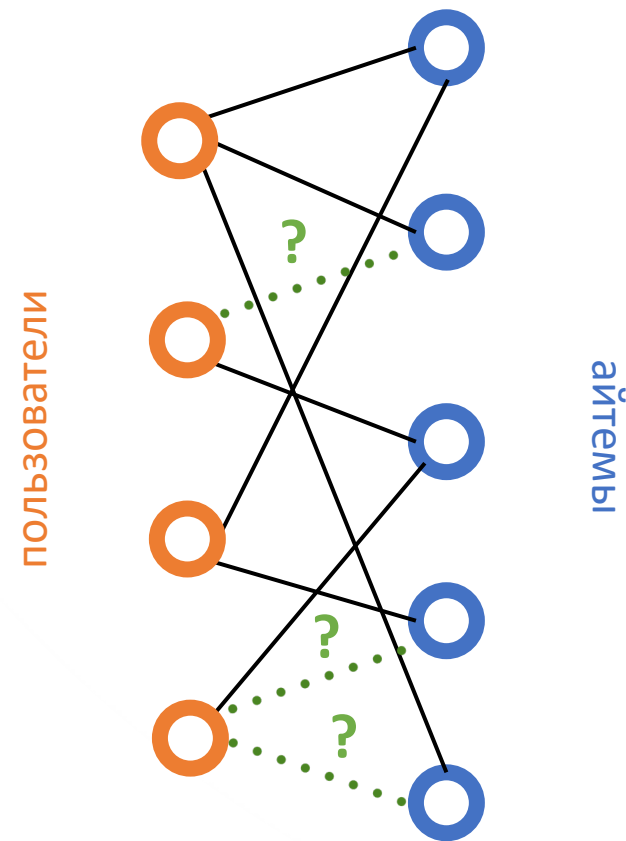
GNN — это естественный подход для достижения обеих целей!

- Нейронная графовая коллаборативная фильтрация (NGCF) [Wang et al. 2019]
- LightGCN [He et al. 2020] (Упрощенная и улучшенная версия NGCF)

Neural Graph Collaborative Filtering

NGCF: Графовая постановка задачи

- Bipartite graph $G = (U \cup V, E)$
- Узлы:
 - $u \in U$ — пользователи
 - $v \in V$ — товары
- Рёбра:
 - взаимодействия (click, rating, purchase)
- **Задача:**
предсказать, появится ли новое ребро (u, v)



Neural Graph Collaborative Filtering

Начальное представление

Каждому узлу ставится в соответствие embedding:

$$h_u^{(0)}, h_v^{(0)}$$

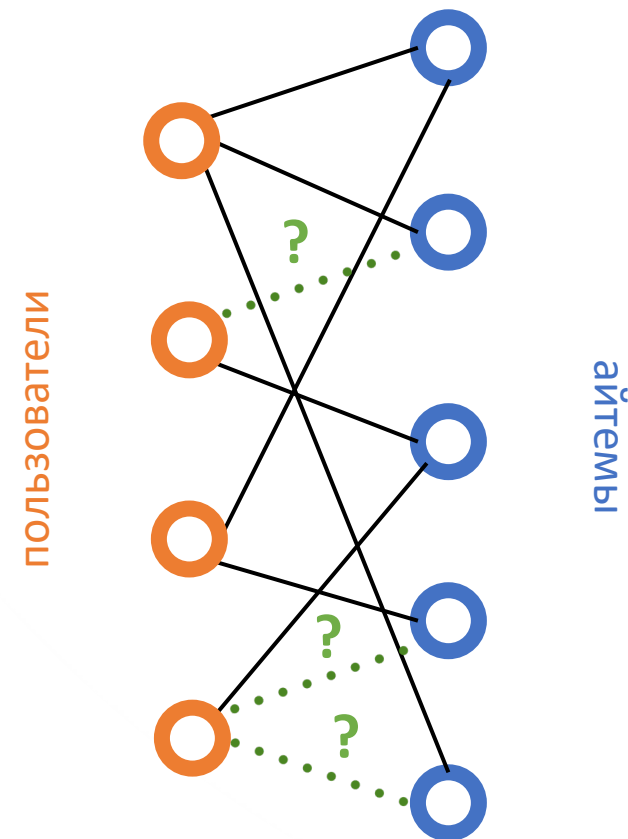
- обучаемые параметры
- аналогичны MF, но это **только старт**

Message Passing:

$$h_u^{(k+1)} = \sigma \left(\sum_{v \in N(u)} \frac{(W_1^k h_v^k + W_2^k (h_v^k \odot h_u^k))}{\sqrt{N(u) \parallel N(v)}} \right)$$

где:

- $N(u)$ — соседи узла
- $\odot$ — поэлементное произведение
- σ — нелинейность (LeakyReLU)



Neural Graph Collaborative Filtering

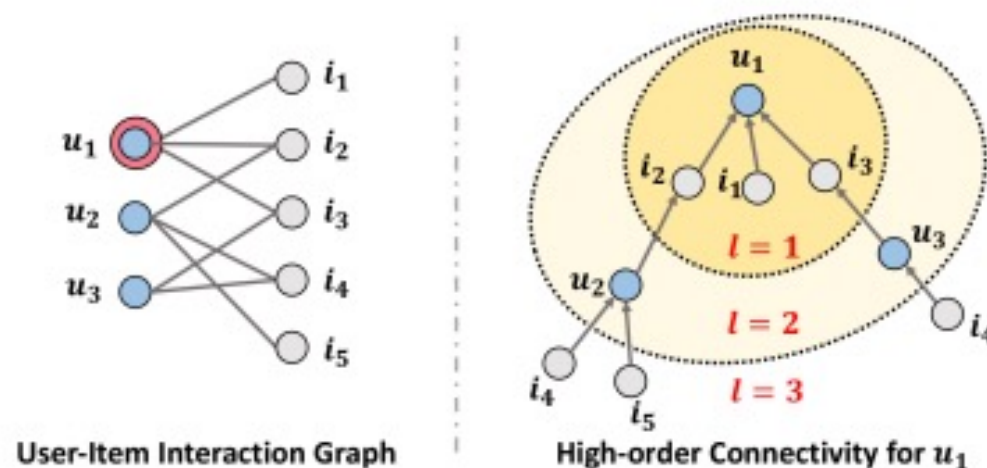
Multi-hop propagation:

Почему несколько слоёв GNN?

- 1 hop:
 - пользователь $\leftrightarrow$ товары
- 2 hops:
 - пользователь $\leftrightarrow$ товары $\leftrightarrow$ другие пользователи
- 3 hops:
 - более сложные паттерны интересов

NGCF объединяет embeddings всех слоёв:

$$h_u = h_u^0 \parallel h_u^1 \parallel \dots \parallel h_u^K$$



Neural Graph Collaborative Filtering

Функция скоринга

Как делается рекомендация

$$\hat{y}_{uv} = h_u^\top h_v$$

- стандартный dot-product
- используется для ранжирования

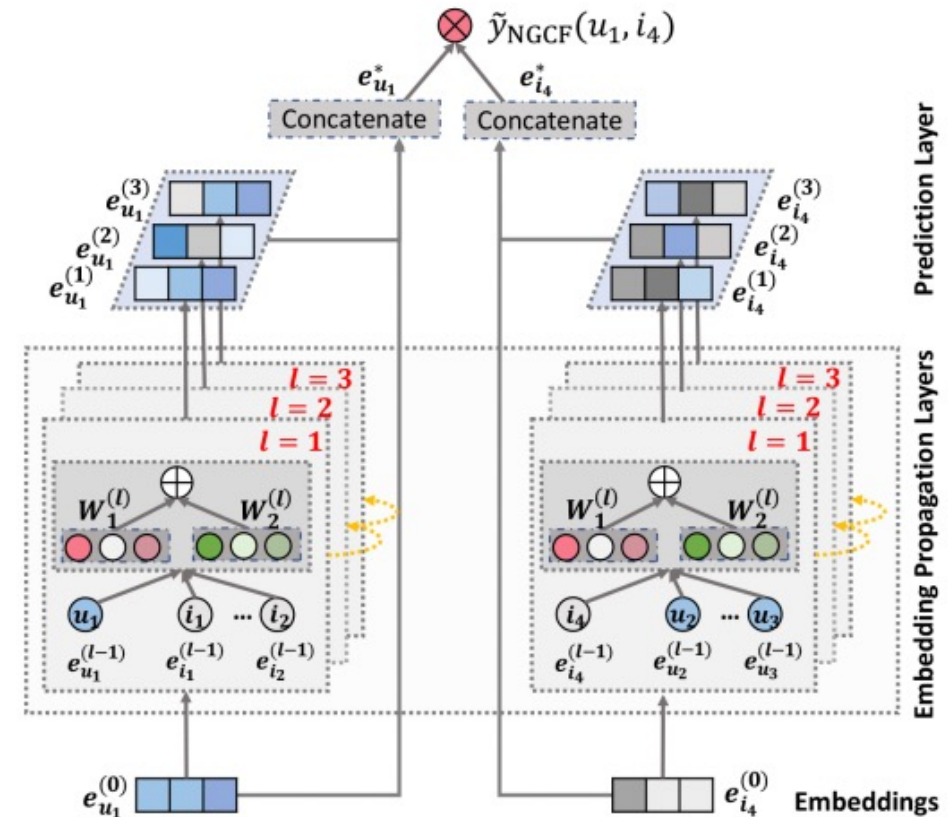
Обучение: BPR Loss (Bayesian Personalized Ranking)

$$L_{BPR} = - \sum_{(u,i,j)} \log \sigma(\hat{y}_{ui} - \hat{y}_{uj})$$

где:

- i — положительный item
- j — отрицательный item

Стандарт для implicit feedback.



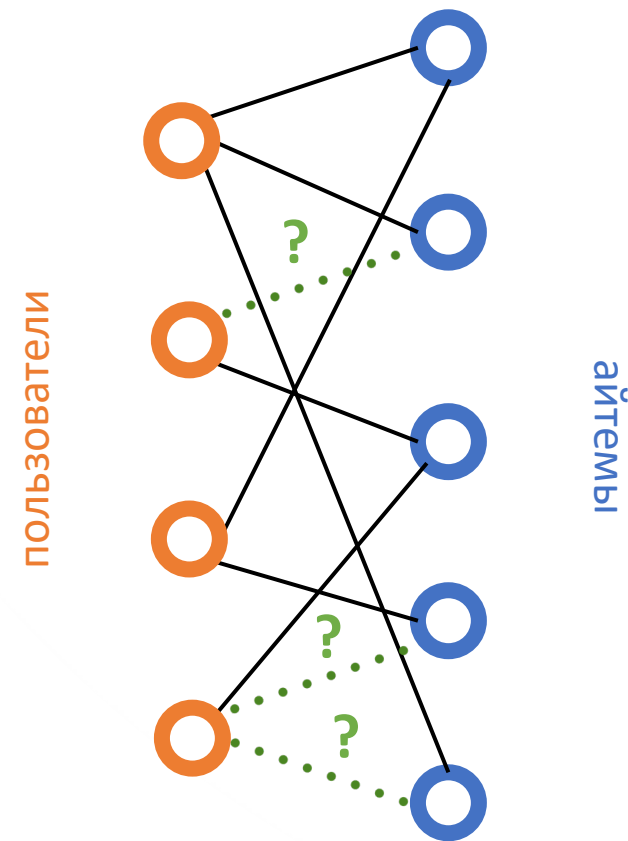
Neural Graph Collaborative Filtering

Результаты статьи

- NGCF > MF, NeuMF
- особенно хорошо: sparse datasets, cold-start сценарии
- выигрывает за счёт структуры графа

Ограничения NGCF

- Почему появились LightGCN и PinSAGE
- тяжёлые нелинейности
- много параметров
- сложная оптимизация
- плохо масштабируется на миллиардные графы



GNN в RecSys

Тема	Статья	Кратко зачем читать	Ссылка
База: RecSys как граф	Graph Neural Networks for Recommender Systems Wu et al., 2020 (Survey)	Обзор всех GNN-подходов в рекомендациях	https://arxiv.org/abs/2005.05673
Переход от MF к GNN	Neural Graph Collaborative Filtering (NGCF) Wang et al., 2019	Первая “настоящая” GNN-модель для RecSys	https://arxiv.org/abs/1905.08108
Практический стандарт	LightGCN He et al., 2020	Упрощённый и эффективный GNN-подход	https://arxiv.org/abs/2002.02126
Индустриальный кейс	PinSAGE Ying et al., 2018	GNN для миллиардных графов (Pinterest)	https://arxiv.org/abs/1806.01973
Гетерогенные графы	Heterogeneous Graph Attention Network (HAN) Wang et al., 2019	Работа с разными типами узлов и рёбер	https://arxiv.org/abs/1903.07293
Temporal RecSys	Temporal Graph Networks (TGN) Rossi et al., 2020	Динамические графы и рекомендации во времени	https://arxiv.org/abs/2006.10637
Self-supervised RecSys	Self-supervised Graph Learning for RecSys Wu et al., 2021	Контрастивное обучение на графах	https://arxiv.org/abs/2101.06868
Knowledge-aware RecSys	KGAT: Knowledge Graph Attention Network Wang et al., 2019	Рекомендации с использованием Knowledge Graph	https://arxiv.org/abs/1905.07854
Cold-start	GraphSAGE Hamilton et al., 2017	Индуктивное обучение на графах	https://arxiv.org/abs/1706.02216
Масштаб и продакшен	Graph Neural Networks in Recommender Systems: A Survey Yu et al., 2022	Практические вопросы масштабирования	https://arxiv.org/abs/2204.03013

GNN B CV

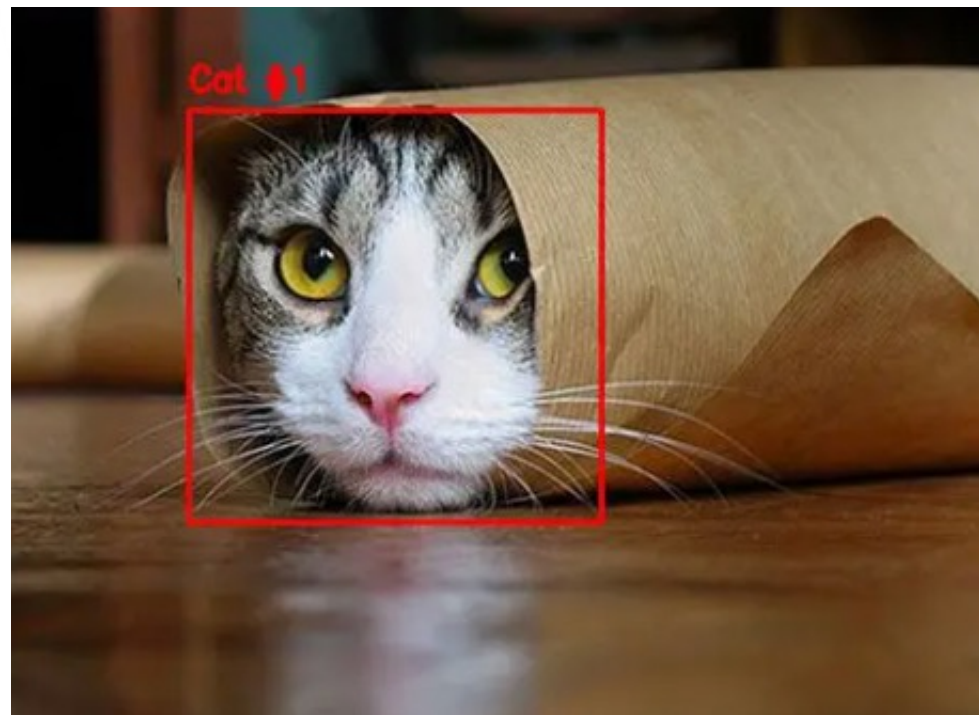
CV

Классическое Computer Vision

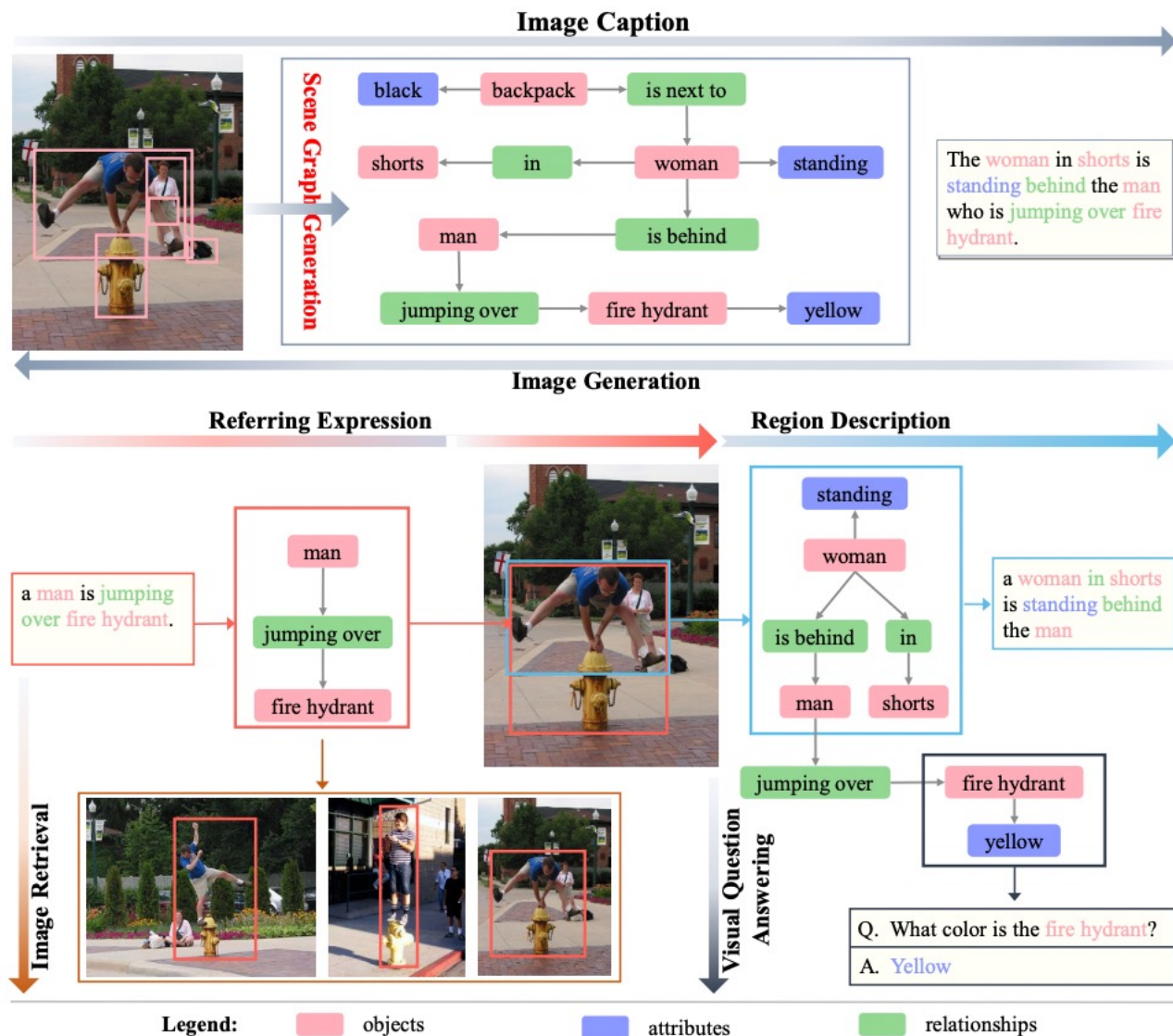
- CNN → пиксели
- локальные receptive fields
- свёртки ≠ отношения

Связи в визуальных данных

- объект ↔ объект
- часть ↔ целое
- соседство
- взаимодействие



Scene Graph Generation



Scene Graph Generation

2D SGG

- построение графа для **обычных изображений**
- методы делятся по типу:
 - feature extraction (как кодировать объекты/отношения)
 - feature fusion (как объединять контекст)
- используются механизмы:
 - **message passing**
 - **attention**
 - **visual translation embedding**

Spatio-temporal SGG

- графы для **видео**
- отношения имеют **временной компонент**, зависят от событий
- отношения меняются во времени

Scene Graph Generation

Основные вызовы, описанные в статье

Авторы выделяют несколько ключевых проблем, с которыми сталкиваются современные методы SGG:

1. Long-tail distribution

- отношения между объектами распределены крайне неравномерно
- большинство объектов/отношений встречается редко
- типично для Visual Genome и других датасетов

2. Большое внутриклассовое разнообразие

- один и тот же тип отношения может выглядеть очень по-разному в разных сценах
- вызывает сложности для детекции и классификации

3. Объединение признаков и контекста

- необходимо эффективно объединять **локальные признаки объектов** и **глобальный контекст сцены**
- методы варьируются от простых конкатенаций до attention и graph-based reasoning

Scene Graph Generation

Применения Scene Graph

Авторы подчёркивают, что scene graph не только формальный граф, но является полезным промежуточным представлением для:

- **image captioning** — генерация описаний
- **visual question answering (VQA)** — вопросы по сцене
- **image retrieval** — поиск по структуре сцены
- **image generation** — генерация сцен по графу
- **referring expression comprehension** — понимание выражений, указывающих на объект в сцене

Основные архитектурные элементы Scene Graph Generation

Scene Graph Generation — это не одна модель, а **комбинация нескольких архитектурных блоков**, каждый из которых решает свою подзадачу.

1. Детекция объектов (Object Detection Backbone)

Первый и обязательный этап SGG — **обнаружение объектов** на изображении.

Типичные компоненты:

- CNN / Vision Transformer backbone
- Region Proposal Network (RPN)
- ROI pooling / ROI Align

На выходе:

- bounding boxes
- визуальные признаки объектов
- классы объектов

Основные архитектурные элементы

Scene Graph Generation — это не одна модель,
а **комбинация нескольких архитектурных блоков**, каждый из которых решает свою подзадачу.

1. Детекция объектов (Object Detection Backbone)

Первый и обязательный этап SGG — **обнаружение объектов** на изображении.

Типичные компоненты:

- CNN / Vision Transformer backbone
- Region Proposal Network (RPN)
- ROI pooling / ROI Align

На выходе:

- bounding boxes
- визуальные признаки объектов
- классы объектов

Основные архитектурные элементы

2. Извлечение признаков отношений (Relationship Feature Extraction)

Для каждой пары объектов (i, j) извлекаются признаки, описывающие **возможное отношение**.

Какие признаки используются

2.1 Визуальные признаки

- CNN-признаки регионов объектов
- признаки области **между** объектами (union box)

2.2 Геометрические признаки

- относительное положение bounding boxes
- расстояние, перекрытие, ориентация

2.3 Семантические признаки

- классы объектов
- вероятности классов

Основные архитектурные элементы

3. Context Modeling (ключевой архитектурный блок)

Это сердце Scene Graph Generation.

Проблема: Отношение между двумя объектами **зависит от всей сцены**, а не только от этой пары.

Основные подходы к modeling context

3.1 Sequential Context Modeling (RNN / LSTM)

- объекты упорядочиваются
- признаки проходят через LSTM / GRU
- контекст распространяется последовательно

Использовалось в ранних работах.

Минусы:

- порядок объектов искусственный
- плохо масштабируется

Основные архитектурные элементы

3. Context Modeling (ключевой архитектурный блок)

3.2 Graph-based Context Modeling (GNN)

Основная идея

- объекты → узлы
- потенциальные отношения → рёбра
- контекст распространяется через message passing

Типичные GNN

- GCN
- Graph Attention Networks
- Relational GNN

Это наиболее естественный подход.

Основные архитектурные элементы

3. Context Modeling (ключевой архитектурный блок)

3.3 Attention-based Context Modeling

- self-attention между объектами
- soft-выбор релевантных взаимодействий
- Transformer-подобные архитектуры

Часто используется **вместе с графами**.

Основные архитектурные элементы

4. Предсказание отношений (Relationship Classification)

Для каждой пары объектов классифицируется тип отношения:

$$o_i, r_{ij}, o_j$$

Архитектурные варианты

- MLP поверх aggregated features
- bilinear / translation-based scoring
- attention-weighted classifiers

Обычно это **многоклассовая классификация**, часто с сильно несбалансированными классами.

Основные архитектурные элементы

5. Работа с long-tail distribution

Проблема:

- few frequent relations (“on”, “near”)
- many rare relations (“riding”, “attached to”)

Архитектурные решения:

- re-weighting loss
- hierarchical relation modeling
- knowledge-guided priors
- debiased training

Основные архитектурные элементы

7. Расширения: видео и 3D

7.1 Spatio-temporal Scene Graphs (Video)

- узлы и рёбра меняются со временем
- temporal edges
- динамические GNN

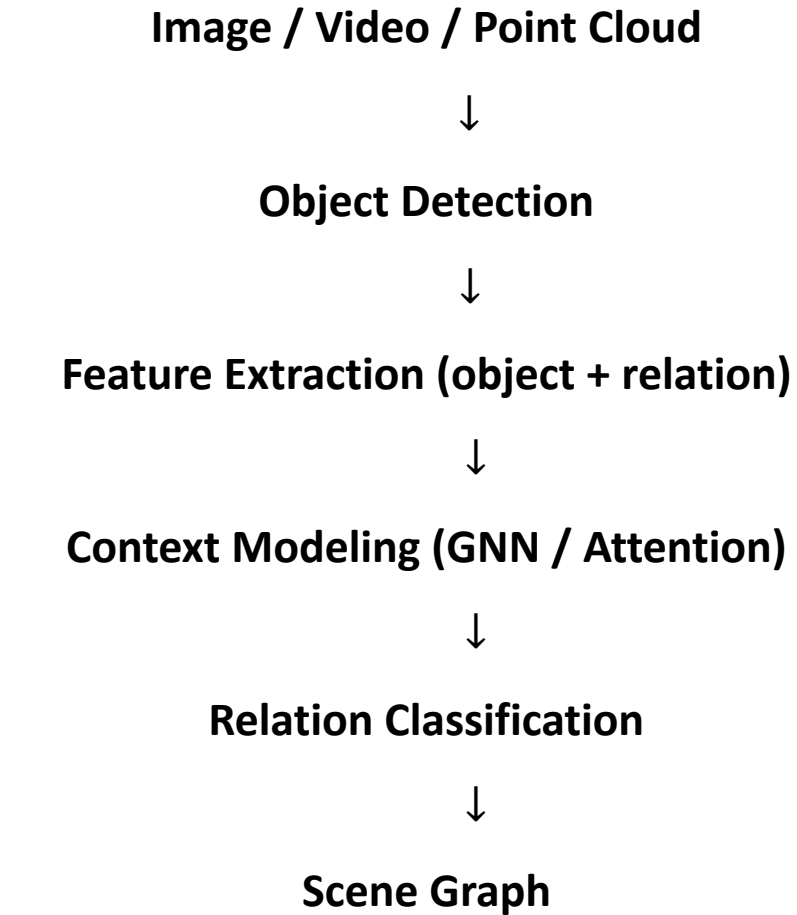
Используется для action understanding и video QA.

7.2 3D Scene Graphs

- узлы = 3D объекты
- отношения = spatial relations в 3D
- граф строится из point cloud / mesh

Часто используется **Point-GNN + GNN reasoning**.

Типовой SGG Pipeline



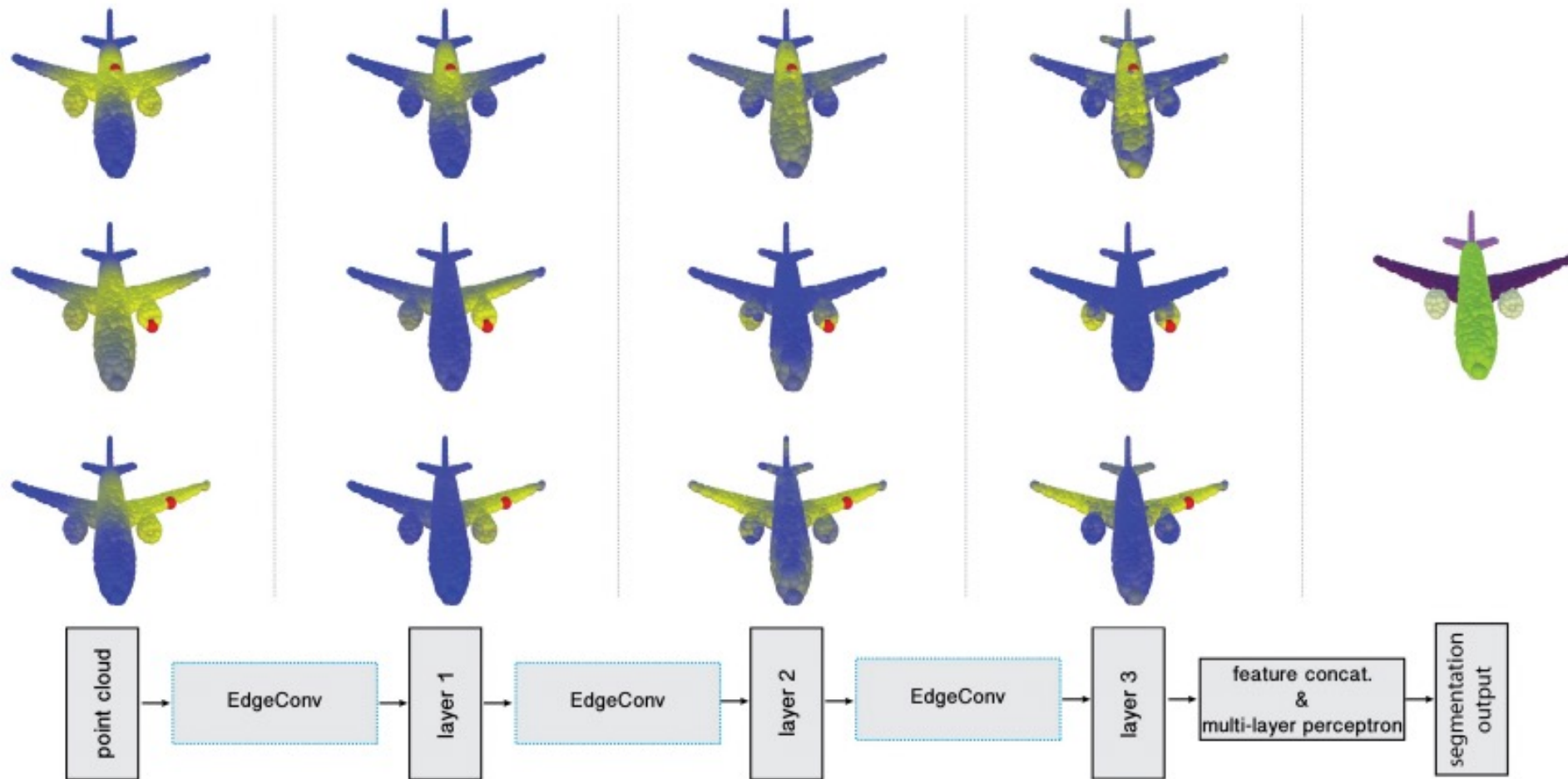
Scene Graph Generation

Применения Scene Graph

scene graph не только формальный граф, но является полезным промежуточным представлением для:

- **image captioning** — генерация описаний
- **visual question answering (VQA)** — вопросы по сцене
- **image retrieval** — поиск по структуре сцены
- **image generation** — генерация сцен по графу
- **referring expression comprehension** — понимание выражений, указывающих на объект в сцене

Point Clouds. Dynamic Graph CNN (DGCNN)

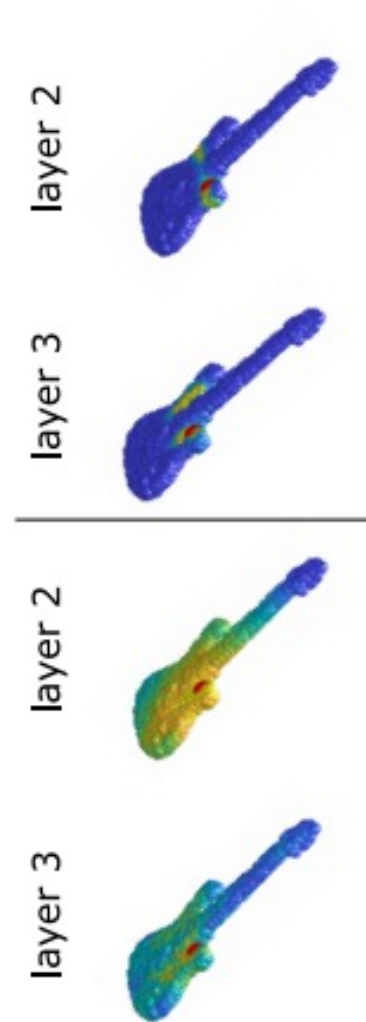


Point Clouds.

Point clouds (облака точек) - это **неструктурированные наборы координат**, получаемые, например:

- с LiDAR-сканеров (автопилоты)
- в 3D-сканировании окружения
- из depth-камер

Они не лежат на регулярной сетке, и для них **нет стандартных свёрток**, как в изображениях; поэтому нужна другая архитектура.



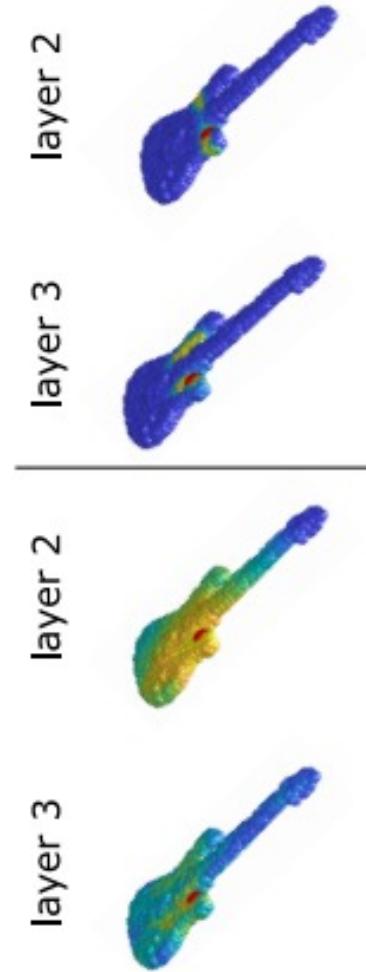
DGCNN

Задачи, которые решает DGCNN

Основной акцент сделан на двух базовых задачах в 3D:

1. **Классификация форм** (shape classification)
2. **Сегментация точек по классам** (per-point semantic segmentation)

Облачные данные поступают напрямую - без вокселизации или проекции на изображения



DGCNN

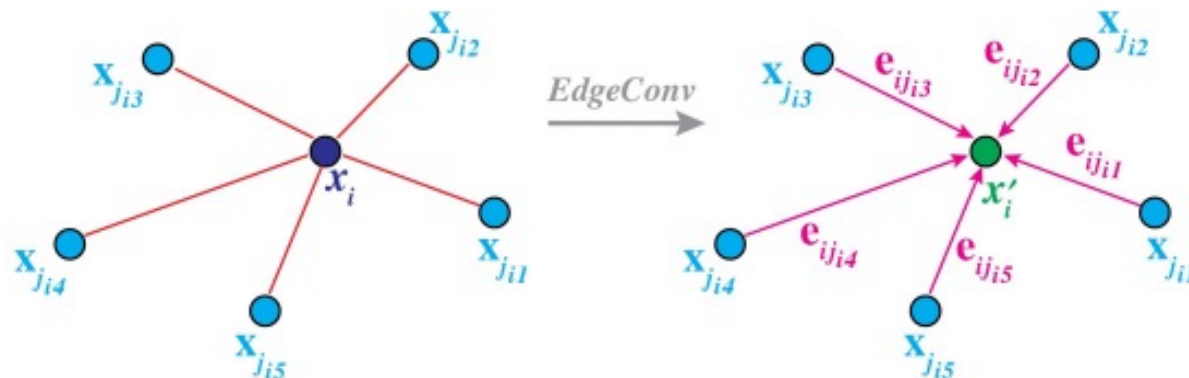
Ключевой строительный блок: EdgeConv

EdgeConv — главный модуль архитектуры DGCNN. Это операция, которая:

- строит граф вокруг каждой точки
- смотрит, кто её ближайшие соседи
- кодирует отношения между точкой и её соседями

Формула EdgeConv звучит как:

операция **агрегирует признаки ребер** между точкой и её соседями, при этом сохраняет инвариантность к порядку точек.



DGCNN

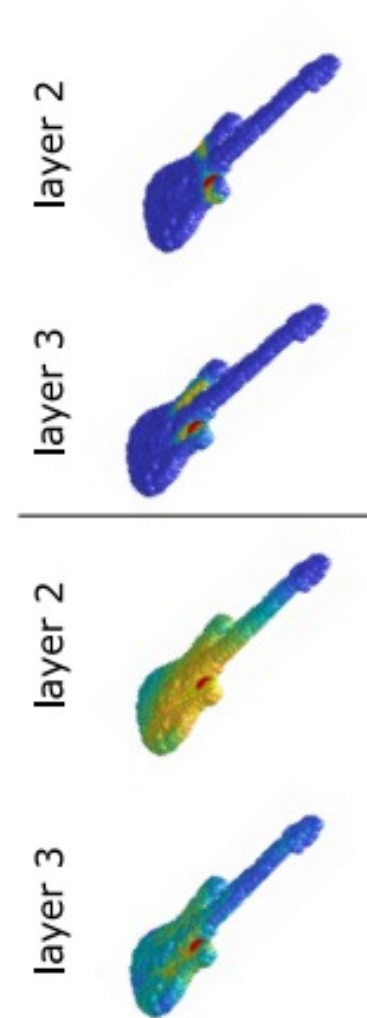
Динамический граф

В отличие от фиксированных графов, DGCNN строит новый граф на каждом уровне сети:

- на входе соседями считаются ближайшие точки по координатам;
- на следующем слое - ближайшие в **фиче-пространстве**, которые могут улавливать семантическую близость.

Это похоже на:

«каждый слой сам учит, какие соседи важны» - и это расширяет receptive field автоматически.



DGCNN

Ключевые инновации работы:

1. EdgeConv — new graph convolution on point clouds:

учитывает локальные отношения и перестраивает граф в каждом слое.

2. Динамический граф строится на основе текущих признаков

что увеличивает семантическую близость соседей.

3. Инвариантность к порядку точек

важно для корректной работы с неструктурированными данными

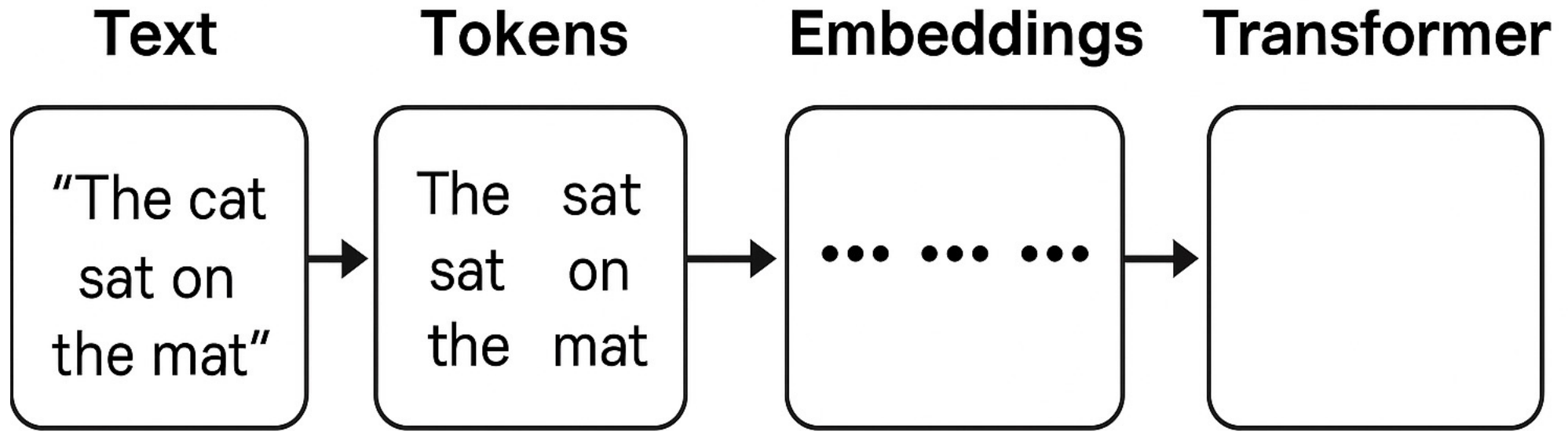
Метод	локальные связи	глобальный контекст
PointNet	нет	да
PointNet++	частично	да
DGCNN (EdgeConv)	да	да

GNN в CV

Тема	Статья	Кратко зачем читать	Ссылка
Обзор (must-read)	A Survey on Graph Neural Networks and Graph Transformers in Computer Vision Bai et al., 2022	Полная карта применения GNN в CV	https://arxiv.org/abs/2209.13232
Point Cloud как граф	Dynamic Graph CNN (DGCNN) Wang et al., 2019	Базовая GNN-архитектура для 3D point cloud	https://arxiv.org/abs/1801.07829
3D Object Detection	Point-GNN Shi et al., 2020	Детекция объектов в LiDAR через графы	https://arxiv.org/abs/2003.01251
Scene Graph Generation	Scene Graph Generation: A Comprehensive Survey Zhu et al., 2022	Обзор scene graph в 2D/3D/видео	https://arxiv.org/abs/2201.00443
Scene Graph Reasoning	Neural Motifs Zellers et al., 2018	Контекст и зависимости между объектами сцены	https://arxiv.org/abs/1711.06640
Visual QA	Graph-based Reasoning over Visual Scenes Teney et al., 2017	GNN для reasoning в Visual Question Answering	https://arxiv.org/abs/1709.04492
Part-based Vision	GCNs for Human Pose Estimation Kipf-inspired, 2018	Суставы как узлы графа	https://arxiv.org/abs/1801.07372
Video & Dynamics	Spatio-Temporal GCNs Yan et al., 2018	Динамические графы для видео	https://arxiv.org/abs/1801.07455
3D Scene Understanding	Learning 3D Semantic Scene Graphs Armeni et al., 2019	Scene graph для 3D сцен	https://arxiv.org/abs/1912.07610
Индустриальный CV	Graph Neural Networks in Autonomous Driving Review, 2021	GNN для автопилотов и робототехники	https://arxiv.org/abs/2106.14587

GNN B NLP

Classic NLP

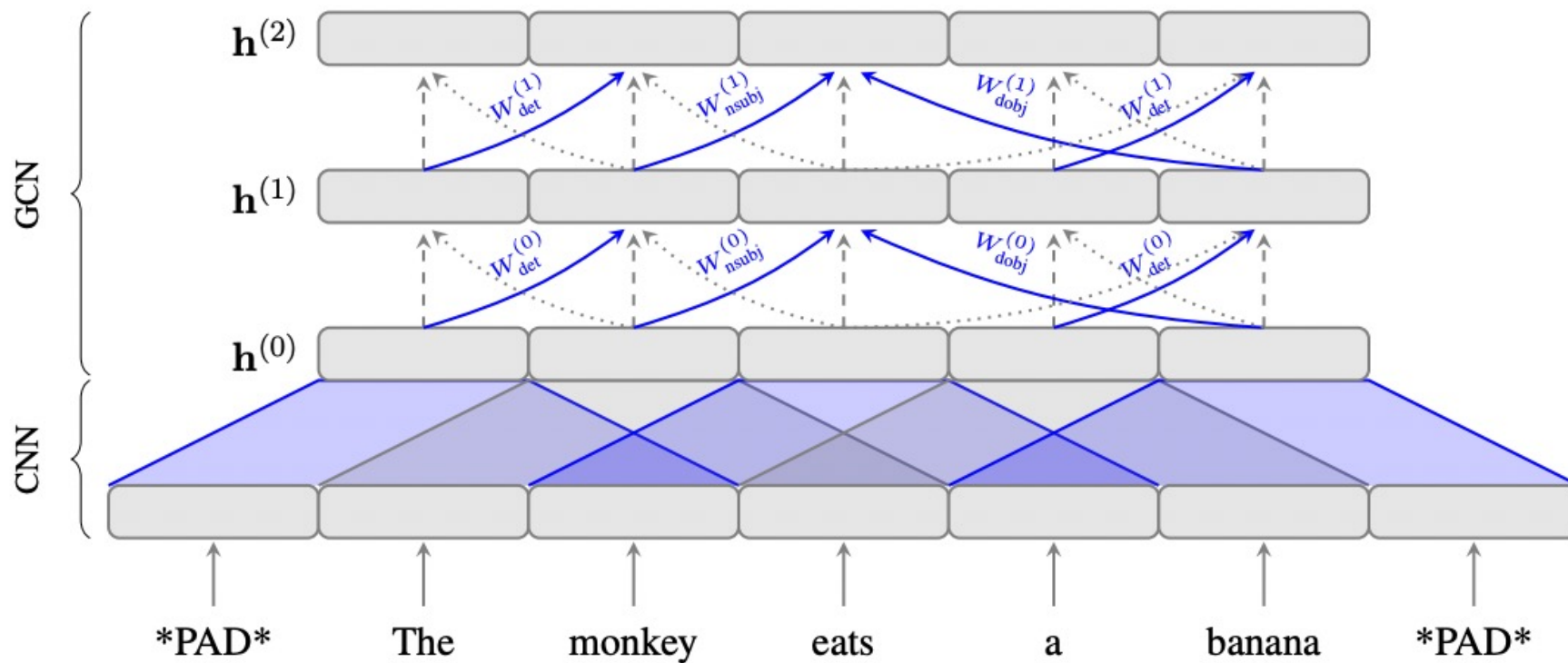


Где последовательность ломается

Типовые сложности:

- **Длинные зависимости:** факт и уточнение разнесены на страницы.
- **Много сущностей:** текст описывает систему объектов, а не одну тему.
- **Multi-hop reasoning:** ответ требует цепочки фактов из разных мест.
- **Много документов:** знание распределено по базе, ссылкам, FAQ, вики.

Какие графы возникают в NLP



Какие графы возникают в NLP

1. Dependency graph: граф синтаксиса

Объяснение:

- Узлы — слова (или токены).
- Рёбра — зависимости (подлежащее, дополнение, модификатор...).
- Это структурирует предложение: “кто что сделал кому”.

Почему это полезно:

- Когда модель должна понимать грамматику, логические связи внутри предложения (особенно в сложных конструкциях).

Важно подчеркнуть:

- Dependency graph — обычно **дерево**, но мы можем рассматривать его как ориентированный граф.
- В реальных системах можно строить dependency graphs парсером и затем делать reasoning/GNN поверх.

Мини-пример:

- “not” и “never” меняют смысл через связь, а не через локальный контекст.

Какие графы возникают в NLP

2. Entity graph: граф сущностей в тексте

Объяснение:

- Узлы — сущности (люди, компании, продукты, места).
- Рёбра — co-occurrence в предложении/абзаце или извлечённые отношения (`works_at`, `located_in`).
- Часто граф строится на корпусе документов, а не на одном тексте.

Почему это полезно:

- Для **поиска**, **кластеризации тем**, **disambiguation** (Apple компания vs apple фрукт), **linking сущностей**.

Какие графы возникают в NLP

3. Knowledge graph: факты и отношения

Объяснение:

- Узлы — сущности.
- Рёбра — типизированные отношения.
- Основная единица: триплет (h, r, t).

Что это даёт:

- Структура, по которой можно делать **multi-hop reasoning**.
- Можно строить ответы как цепочку фактов.

Важно: KG — не всегда “идеальная база знаний”. Он может быть шумным, неполным, автоматически извлечённым из текста.

Какие графы возникают в NLP

4. Document graph: граф документов и фрагментов

Объяснение:

- Узлы: документы / chunks / абзацы / страницы.
- Рёбра:
 - ссылки/цитирования
 - общие сущности
 - тематические связи (например, kNN по embedding + фильтры)

Почему важно:

- Это мост к RAG.
- Для retrieval нам часто нужен **граф корпуса**, а не только embedding поиск.

Что именно делают GNN в NLP

1. Message passing = распространение релевантности

Объяснение простыми словами:

- Каждый узел получает сообщения от соседей.
- После нескольких шагов узел “знает” не только о себе, но и о своём окружении.

Применение в NLP:

- Сущность “заряжается” контекстом через соседние сущности.
- Документ “заряжается” информацией через связанные документы.

Ключ:

“В NLP GNN часто играет роль слоя, который добавляет структуру к эмбедингам, полученным из Transformer.”

Что именно делают GNN в NLP

2. Типовая гибридная архитектура: Transformer + GNN

Шаблон:

1. Transformer кодирует текст → эмбединги токенов/сущностей/чанков
2. Строим граф (entities/docs/relations)
3. GNN делает propagation → улучшенные эмбединги
4. Решаем задачу: классификация/QA/retrieval

Почему гибрид работает:

- Transformer сильнее в “локальной семантике”.
- GNN сильнее в “структуре и многошаговых связях”.

Где GNN даёт выигрыш

GNN особенно полезны, когда:

- есть **явные связи** (ссылки, KG, цитирование)
- нужен **multi-hop reasoning**
- данные **разреженные**, а связи информативны

GNN может быть лишним, если:

- задача чисто локальная (классификация короткого текста)
- граф построен плохо (много шума)

Вопрос аудитории:

“Если граф построен из мусорных связей, будет ли GNN помогать?”

Graph-based QA: reasoning по графу

Почему QA — это не просто поиск

Пример:

- Вопрос: “Какая компания владеет продуктом X?”
- Факты:
 - X разработан стартапом В
 - В купила компания А

Ответ требует 2 шага.

Вывод:

- Даже идеальный “top-1” абзац может не содержать весь ответ.
- Нужно связать куски знаний.

Типовой пайплайн graph QA

1. Распознать сущности в вопросе
2. Найти кандидатов в KG/корпусе
3. Собрать подграф
4. Запустить GNN: распространить релевантность
5. Выбрать путь/узлы как доказательство
6. Сгенерировать ответ

От RAG к Graph-RAG

Классический RAG:

Быстро:

- embed query
- retrieve top-k chunks
- LLM generates

Структурный retrieval



Graph-RAG

Graph-RAG pipeline

1. Разбить документы на chunks
2. Выделить сущности (NER + linking)
3. Построить граф: (chunk–entity, entity–entity (co-occurrence / relation extraction / KG), chunk–chunk (ссылки, тематическая близость))
4. Инициализировать эмбединги:
 - chunk embeddings (например, от sentence transformer)
 - entity embeddings (из описаний или через LM)
5. Прогнать GNN:
 - обновить эмбединги с учётом структуры
6. Retrieval:
 - стартовые узлы: top-k по similarity
 - расширение: k-hop neighborhood / personalized PageRank / learned scoring
7. Собрать контекст → LLM → ответ
8. (Опционально) вернуть “evidence graph” (объяснимость)

GNN в NLP. Источники

Тема	Статья	Кратко зачем читать	Ссылка
Text → Graph	Graph Convolutional Networks for Text Classification (TextGCN) Yao et al., 2019	Как из корпуса текстов построить граф (слова + документы) и применять GCN	https://arxiv.org/abs/1904.01274
Синтаксис как граф	Graph Convolutional Encoders for Syntax-aware NMT Bastings et al., 2017	Dependency graph + GCN для улучшения NLP-моделей	https://arxiv.org/abs/1704.04675
Relation Extraction	Graph Convolution over Pruned Dependency Trees Zhang et al., 2018	Извлечение отношений через GNN поверх dependency-графов	https://arxiv.org/abs/1809.03646
Document-level Graphs	Global Graph Modeling for Relation Extraction Guo et al., 2019	Граф сущностей и упоминаний на уровне документа	https://arxiv.org/abs/1906.03388
Knowledge Graphs + GNN	Modeling Relational Data with Graph Convolutional Networks (R-GCN) Schlichtkrull et al., 2018	Базовая GNN-модель для Knowledge Graphs и reasoning	https://arxiv.org/abs/1703.06103
Multi-hop QA	Reasoning over Semantic Graphs for Multi-Hop QA Sun et al., 2020	Построение семантического графа под конкретный вопрос	https://arxiv.org/abs/2003.13873
Text Generation / Summarization	Graph Structured Network for Document Modeling Liu et al., 2019	Использование графов перед генерацией текста	https://arxiv.org/abs/1904.11369
GNN + Transformer	Composition-based Multi-Relational GCN (CompGCN) Vashishth et al., 2019	Совмещение графов и нейронных языковых моделей	https://arxiv.org/abs/1904.05255
Graph-aware LM	KEPLER: Knowledge Embedding + LM Wang et al., 2021	Связывает Knowledge Graph и language models — полезно для Graph-RAG	https://arxiv.org/abs/2009.00710
Survey (обзор)	A Survey on GNNs and Graph Transformers in NLP Bai et al., 2022	Полная карта области: задачи, методы, тренды	https://arxiv.org/abs/2206.14201
Scene Graph → NLP bridge	Scene Graph Generation: A Comprehensive Survey Zhu et al., 2022	Связь CV-графов, reasoning и NLP-задач	https://arxiv.org/abs/2201.00443